■ Barber Queue — Development Journal

Date: 16 September 2026 | Project: Barber Queue / THE CUT

Session goal: Build the foundation of a real barber queue system that can be shown to and tested by a barber.

1. Project Foundation

What was done: Created the Barber Queue project in VS Code with three main files: index.html, style.css, and script.js.

Explanation: HTML controls structure and content, CSS controls appearance and layout, and JavaScript controls interactions and logic.

Insight: Separating responsibilities makes the project easier to understand, debug, and expand.

What should be noted: Keep HTML, CSS, and JavaScript organized rather than putting everything into one file as the project grows.

2. THE CUT Opening Screen

What was done: Created the opening screen with THE CUT, the tagline “Your time. Your turn. Your cut.”, and Client/Barber choices.

Explanation: The application determines what type of user is using it and presents a different interface for each role.

Insight: This moved the idea from a basic webpage toward a business tool.

What should be noted: Role selection is currently prototype navigation, not real authentication or security.

3. Premium Barber Design

What was done: Built a mobile-first black and warm-gold barber aesthetic with clean typography and a premium visual direction.

Explanation: CSS creates the visual identity and makes the prototype feel closer to a real product.

Insight: If a real business may eventually pay for the product, presentation matters alongside functionality.

What should be noted: Design should support usability rather than adding effects only because they look cool.

4. Client / Barber Navigation

What was done: Added navigation so the Client button opens the client app, the Barber button opens the barber app, and back buttons return to role selection.

Explanation: JavaScript event listeners detect clicks and use the hidden class to control which section is visible.

Insight: Core application pattern: user action → JavaScript detects it → application changes state.

What should be noted: Both interfaces are currently sections of one webpage.

5. Join Queue

What was done: The Join the Queue button opens a customer form where the customer enters a name and chooses a service.

Services: Haircut; Hair + Beard; Beard; Line Up.

Insight: The queue needs structured information, not just a name. Future records can include position, waiting time, status, and time joined.

6. Confirm & Join Queue

What was done: The confirmation button validates the customer's name and selected service, creates a customer object, and adds it to the queue.

Code concept: A customer object stores information such as id, name, service, and position. queue.push(customer) adds the customer to the queue.

Insight: This was the transition from building an interface to managing actual application data.

What should be noted: The current queue is temporary browser memory; persistence is a later stage.

7. Customer Ticket

What was done: After joining, the customer receives a ticket showing queue position, name, service, and estimated waiting time.

Explanation: The current prototype estimates waiting time using roughly 30 minutes per position.

Insight: The estimate is temporary. Different services should eventually have different durations.

What should be noted: The current waiting-time calculation is not production-ready.

8. Current Queue

What was done: Added a simple queue display showing every customer in order, with position, name, and service.

Explanation: JavaScript loops through every customer in the queue and creates a queue item for each one.

Insight: The display was simplified after a problem made it appear that earlier customers were disappearing.

What should be noted: The newest customer's ticket is displayed at the top, while all customers remain in the queue.

9. Multiple Customers

What was done: After one customer joins, the form clears, the Join Queue button returns, and another customer can be added without replacing the previous customer.

Example: John — Haircut; Mike — Hair + Beard; David — Line Up.

Insight: The array queue[] is the first version of the application's queue data structure.

What should be noted: It is not a real database yet. Different phones/browsers do not share the same queue.

10. Current Project State

Working: Opening screen; Client role; Barber role; Client dashboard; Join queue; name input; service selection; confirmation; ticket; queue position; estimated wait; multiple customers; current queue; simplified queue design.

Not finished: Barber dashboard functionality; shared live queue; start/complete haircut; remove customer; persistent storage; service-specific timing; production authentication.

11. Current Architecture

Flow: THE CUT → Client or Barber → Client joins queue → customer data → queue[] → queue display.

Key concept: queue[] is currently the central in-memory data structure.

12. Biggest Technical Limitation

Current limitation: let queue = [] means the queue exists only during the current page session.

Refresh behavior: Refreshing the page resets the queue.

Multiple devices: Another phone/browser currently has a separate queue.

Future architecture: Client phone → Server → Database → Barber phone.

Insight: A real multi-user product will need a backend/database so everyone sees the same live queue.

13. Biggest Product Insight

Core problem: Customers spend too much time waiting in long barber queues.

Product value: The application should reduce uncertainty and organize the barber's workflow.

Future flow: Customer joins → gets position → sees estimated wait → barber serves next customer → customer can be notified → customer arrives at the right time.

14. Lessons From Today's Coding

Lesson 1: Don't confuse the interface with the data. Something disappearing visually does not necessarily mean the data disappeared.

Lesson 2: Build in small stages: screen → navigation → form → customer → queue → multiple customers → cleaner display.

Lesson 3: Test after every major change: one customer, multiple customers, empty inputs, different services, and returning home.

Lesson 4: A prototype does not need everything. Payments, accounts, notifications, analytics, AI, databases, and advanced booking can come later.

15. What Should Be Noted for the Next Session

Immediate target: Build the Barber Dashboard.

Barber actions: View the same customers; see who is first; start a haircut; complete a haircut; remove a customer; automatically move the remaining queue forward.

After that: Add service-specific durations, better waiting estimates, localStorage persistence, refresh protection, testing, and polish.

Business validation: Show the prototype to the barber and ask what would actually be useful, what they would change, whether customers would use it, and what they would pay for.

16. Final Session Insight

Milestone: The project moved from “I have an idea for a barber app” to “I have a working prototype that can accept customers and organize them into a queue.”

Next goal: Do not make it huge. Make the existing idea genuinely useful to one barber.